

M2 Génie de l'Informatique Logicielle
UE Full Stack 2025-2026

Rapport - TP3

Auteur :
Matisse SENECHAL



Table des matières

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	Périmètre technique	2
2	Modernisation et Maintenance (E_TEC)	3
2.1	Montée de version des Frameworks	3
2.2	Audit de sécurité et vulnérabilités	3
3	Architecture et Conteneurisation (E_PRO, E_ARC)	5
3.1	Configuration Docker et Healthchecks	5
3.2	Alimentation des données (E_PRO_20)	6
4	Développement et Couverture des Exigences	7
4.1	Gestion des Boutiques (Domaine BTQ)	7
4.1.1	Cycle de vie et Validation (CRUD)	7
4.1.2	Recherche Avancée et Elasticsearch	8
4.2	Gestion des Produits (Domaine PRD)	9
4.2.1	Gestion Monétaire (E_PRD_15)	9
4.2.2	Internationalisation (E_PRD_80)	9
4.2.3	Association et Catégorisation	10
4.3	Gestion des Catégories (Domaine CAT)	10
5	Difficultés rencontrées et Solutions (E_PRO_60)	11
5.1	Synchronisation PostgreSQL / Elasticsearch	11
5.2	Gestion des types de données (Horaires)	11
6	Améliorations UX et Qualité (E_AME)	12
6.1	Responsive Design (E_AME_30)	12
6.2	Qualité de Code	12
7	Conclusion	13

Chapitre 1

Introduction

1.1 Contexte et Objectifs

Ce document constitue le rapport descriptif de mes travaux personnels réalisés dans le cadre du projet Full Stack, conformément à l'exigence `E_PRO_60`. Ce projet vise la refonte complète d'une application existante de gestion de boutiques et de produits. Les objectifs principaux incluent la modernisation de la stack technique (`E_TEC`), la sécurisation des dépendances, l'amélioration de l'expérience utilisateur et l'intégration d'un moteur de recherche performant via Elasticsearch.

1.2 Périmètre technique

Le système repose sur une architecture API REST avec une base de données relationnelle PostgreSQL et une base NoSQL Elasticsearch. L'ensemble est conteneurisé via Docker pour assurer la portabilité et la facilité de déploiement.

Chapitre 2

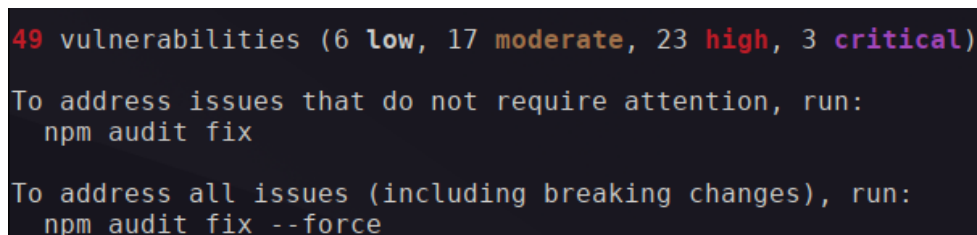
Modernisation et Maintenance (E_TEC)

2.1 Montée de version des Frameworks

Conformément aux exigences E_TEC_50 à E_TEC_80, j'ai procédé à la mise à jour des versions LTS des frameworks (Spring Boot pour le back-end, React pour le front-end) ainsi que de leurs dépendances respectives. Cette étape cruciale garantit la pérennité de l'application et l'accès aux dernières fonctionnalités de sécurité.

2.2 Audit de sécurité et vulnérabilités

Avant la migration, l'analyse des dépendances révélait plusieurs vulnérabilités critiques. Suite à la montée de version et au nettoyage des bibliothèques obsolètes, j'ai considérablement réduit la surface d'attaque.



```
49 vulnerabilities (6 low, 17 moderate, 23 high, 3 critical)
To address issues that do not require attention, run:
  npm audit fix
To address all issues (including breaking changes), run:
  npm audit fix --force
```

FIGURE 2.1 – Rapport npm avant la mise à jour (Exemple)

Comme l'illustre le rapport ci-dessous, l'installation des nouvelles dépendances présente un bilan nettement plus optimiste :

```
node_modules/resolve-url-loader

webpack-dev-server <=5.2.0
Severity: moderate
webpack-dev-server users' source code may be stolen when
GHSA-9jgg-88mc-972h
webpack-dev-server users' source code may be stolen when
fix available via `npm audit fix --force`
Will install react-scripts@0.0.0, which is a breaking cha
node_modules/webpack-dev-server

9 vulnerabilities (3 moderate, 6 high)
To address all issues (including breaking changes), run:
```

FIGURE 2.2 – Rapport npm après correction des vulnérabilités

Chapitre 3

Architecture et Conteneurisation (E_PRO, E_ARC)

3.1 Configuration Docker et Healthchecks

Pour répondre aux exigences d'architecture (E_ARC_60) et de configuration (E_PRO_170), j'ai mis à jour les fichiers `Dockerfile` (Front et Back) ainsi que le `docker-compose.yml`.

Une attention particulière a été portée à l'orchestration des services. J'ai défini des vérificateurs de santé (*healthchecks*) pour Elasticsearch et PostgreSQL. L'API ne démarre désormais que lorsque ces bases de données sont pleinement opérationnelles ("healthy").

```
1 services:
2   db:
3     image: postgres:15
4     healthcheck:
5       test: ["CMD-SHELL", "pg_isready -U postgres"]
6       interval: 10s
7       timeout: 5s
8       retries: 5
9     # ... volumes et ports
10
11   elasticsearch:
12     image: docker.elastic.co/elasticsearch/elasticsearch:7.17.25
13     environment:
14       - discovery.type=single-node
15     healthcheck:
16       test: ["CMD-SHELL", "curl -f http://localhost:9200/_cluster/health
17         || exit 1"]
18       interval: 10s
19       timeout: 5s
20       retries: 5
21
22   api:
23     depends_on:
24       db:
25         condition: service_healthy
26       elasticsearch:
27         condition: service_healthy
28     # ...
```

Listing 3.1 – Extrait du `docker-compose.yml` avec Healthchecks

3.2 Alimentation des données (E_PRO_20)

Afin de faciliter les tests et respecter l'exigence E_PRO_20, j'ai mis en place des scripts SQL montés via Docker. Au lancement, le système est automatiquement peuplé avec au moins 10 boutiques, 100 produits et 10 catégories, permettant de valider immédiatement la pagination et la recherche.

Chapitre 4

Développement et Couverture des Exigences

4.1 Gestion des Boutiques (Domaine BTQ)

La gestion des boutiques constitue le cœur de l'application. Ainsi on a une API REST complète permettant de satisfaire les exigences E_BTQ_10 à E_BTQ_80.

4.1.1 Cycle de vie et Validation (CRUD)

Le système permet la création (E_BTQ_10), la modification (E_BTQ_20) et la suppression (E_BTQ_30) des boutiques. Une attention particulière a été portée à la structure des données :

- **Horaires d'ouverture** : Contrairement à une simple chaîne de caractères, les horaires sont gérés comme une collection d'objets structurés (Jour, Heure début, Heure fin).
- **Validation (E_FIX_10)** : Pour garantir la cohérence des données, j'ai implémenté un algorithme de vérification côté serveur. Lors de la soumission d'un formulaire, le système détecte toute intersection temporelle (chevauchement) entre deux plages horaires pour un même jour et rejette la requête le cas échéant.


```

348 /**
349  * Analyse une liste d'horaires journaliers pour détecter les conflits temporels.
350  *
351  * @throws BadRequestException En cas de chevauchement détecté.
352  */
353 private void checkForOverlap(List<OpeningHoursShop> dayOpeningHours) throws BadRequestException {
354     List<OpeningHoursShop> sortedHours = dayOpeningHours.stream()
355         .sorted(Comparator.comparing(OpeningHoursShop::getOpenAt))
356         .toList();
357
358     for (int i = 0; i < sortedHours.size() - 1; i++) {
359         OpeningHoursShop current = sortedHours.get(i);
360         OpeningHoursShop next = sortedHours.get(i + 1);
361
362         if (isOverlapping(current, next)) {
363             throw new BadRequestException(
364                 String.format(
365                     format: "Les horaires d'ouverture se chevauchent pour le jour %d : %s et %s",
366                     current.getDay(),
367                     current,
368                     next
369                 )
370             );
371         }
372     }
373 }
374
375 /**
376  * Teste l'intersection temporelle entre deux plages horaires.
377  *
378  * @return true si l'heure de fermeture du premier créneau est postérieure à l'ouverture du second.
379  */
380 private boolean isOverlapping(OpeningHoursShop hours1, OpeningHoursShop hours2) {
381     return !hours1.getCloseAt().isBefore(hours2.getOpenAt());
382 }

```

FIGURE 4.1 – Interface de recherche multicritère (Elasticsearch)

4.1.2 Recherche Avancée et Elasticsearch

Pour répondre aux exigences de recherche (E_BTQ_50, E_BTQ_60) et de tri (E_BTQ_70), l'architecture hybride PostgreSQL / Elasticsearch prend tout son sens :

- **Recherche Plein Texte** : L'intégration d'Elasticsearch permet une recherche performante sur le nom des boutiques, accessible via le paramètre d'URL `search`.
- **Filtres Combinés** : Le contrôleur REST accepte une combinaison de paramètres (dates de création, statut de congé) pour affiner les résultats dynamiquement.
- **Données Calculées** : Conformément à E_BTQ_50, la réponse de l'API inclut des agrégats calculés (nombre de produits, nombre de catégories distinctes) sans impacter les performances, grâce à l'utilisation de formules Hibernate ou de l'indexation.

Exemple : `/api/v1/shops?createdAfter=2025-01-10&search=test&size=9`

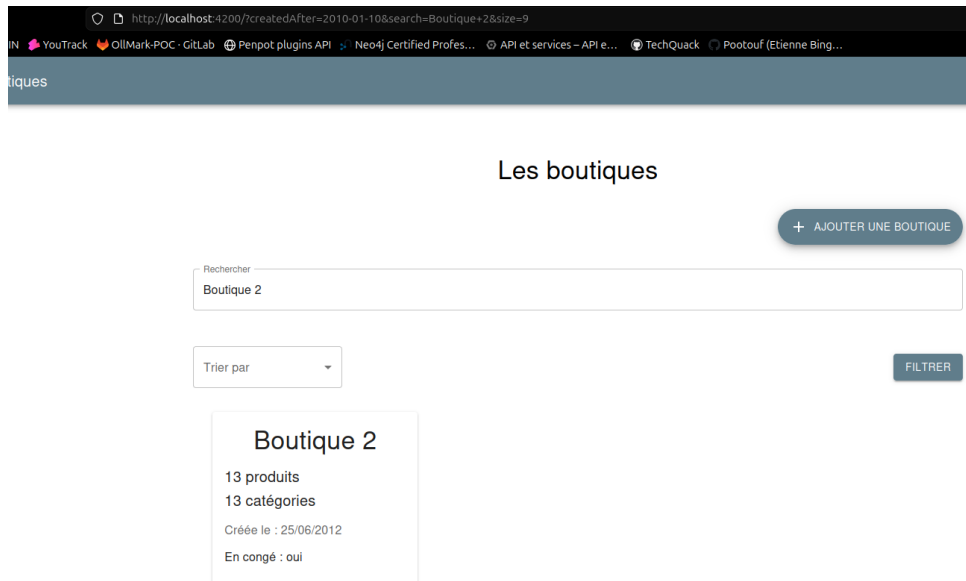


FIGURE 4.2 – Interface de recherche multicritère (Elasticsearch)

4.2 Gestion des Produits (Domaine PRD)

Le module produit a été refondu pour intégrer des contraintes monétaires et linguistiques strictes.

4.2.1 Gestion Monétaire (E_PRD_15)

La manipulation des prix est un point critique. Pour éviter les erreurs d'arrondi inhérentes aux nombres flottants (type `float` ou `double`) :

- **Stockage** : Les prix sont persistés en base de données sous forme d'entiers représentant les centimes.
- **Affichage** : Une conversion automatique (division par 100) est effectuée lors de la sérialisation JSON vers le client, affichant ainsi des Euros à l'utilisateur.
- **Saisie** : L'opération inverse est réalisée lors de la soumission du formulaire, rendant le processus totalement transparent pour le gestionnaire.

FIGURE 4.3 – Formulaire produit : Saisie en Euros, stockage en Centimes

4.2.2 Internationalisation (E_PRD_80)

L'application supporte désormais le multilinguisme pour les champs textuels (Nom, Description). Techniquement, l'entité `Product` possède une relation *One-to-Many* vers

une entité `LocalizedProduct`. Le front-end sélectionne automatiquement la traduction correspondant à la langue de l'interface, ou se rabat sur une langue par défaut si nécessaire.

4.2.3 Association et Catégorisation

Le système permet d'associer un produit à une ou plusieurs catégories (`E_PRD_50`) et de filtrer les produits d'une boutique par catégorie (`E_PRD_70`), facilitant ainsi la gestion des inventaires complexes.

4.3 Gestion des Catégories (Domaine CAT)

Bien que moins complexe que les boutiques, la gestion des catégories est essentielle pour la classification. J'ai implémenté les exigences `E_CAT_10` à `E_CAT_70` :

- **Administration** : Les utilisateurs peuvent créer, éditer et supprimer des catégories.
- **Navigation** : Une vue liste paginée (`E_CAT_50`) permet de visualiser l'ensemble des catégories disponibles.
- **Relation** : L'intégrité référentielle est assurée lors de l'association avec les produits (`E_CAT_40`).

Chapitre 5

Difficultés rencontrées et Solutions (E_PRO_60)

5.1 Synchronisation PostgreSQL / Elasticsearch

Difficulté : Assurer que les données modifiées dans la base relationnelle soient immédiatement reflétées dans le moteur de recherche pour les filtres complexes.

Solution : Implémentation d'un mécanisme synchrone dans le service Spring Boot qui indexe les documents dans Elasticsearch lors des opérations d'écriture (Create/Update).

5.2 Gestion des types de données (Horaires)

Difficulté : La manipulation des objets `LocalTime` et la validation des conflits d'horaires ont posé des problèmes de sérialisation JSON.

Solution : Utilisation d'annotations Jackson spécifiques pour le formatage "HH:mm:ss" et écriture d'un algorithme de détection d'intersections temporelles.

Chapitre 6

Améliorations UX et Qualité (E__AME)

6.1 Responsive Design (E__AME__30)

L'interface client a été retravaillée pour s'adapter aux différents terminaux. Les grilles de produits et les formulaires utilisent désormais des propriétés de mise en page flexibles (Flexbox/Grid via Material UI).

6.2 Qualité de Code

Des efforts de refactoring ont été menés (E__AME_10) pour normaliser le nommage et extraire la logique métier dans des services dédiés, améliorant la maintenabilité.

Chapitre 7

Conclusion

Le TP3 Full-Stack m'a permis de mettre en pratique des compétences essentielles en développement et en maintenance d'applications web. Parmi les principales réalisations, j'ai pu retenir :

- La **mise à jour réussie** de la stack technique, améliorant sécurité et performance.
- L'**intégration complète d'Elasticsearch**, outil qui m'était totalement inconnu et dont j'ai pu comprendre l'intérêt de son utilisation pour de la recherche textuelle puissante.
- La résolution de problèmes de logique métier (horaires, prix en centimes).
- La **conteneurisation robuste** avec Docker Compose et l'alimentation automatique des données de test.

L'application livrée est conforme aux standards demandés, et prête à être déployée.